

Guía de Práctica de Laboratorio N°7: React Defensivo y Resiliencia en Aplicaciones Web

La programación defensiva es una filosofía de diseño que asume que los fallos son inevitables. En el desarrollo con React, esta disciplina nos obliga a anticipar estados de error, inconsistencias en las APIs y fluctuaciones en la red para garantizar que la interfaz de usuario (UI) sea resiliente. Un sistema robusto no es aquel que nunca falla, sino aquel que sabe cómo fallar con elegancia y recuperarse sin degradar la experiencia del usuario.

En este laboratorio, transformaremos una aplicación frágil en una arquitectura preparada para producción, aplicando técnicas de refactorización de servicios, diseño de UI centrado en la estabilidad y validación de datos mediante Inteligencia Artificial.

Objetivos de Aprendizaje

1. **Refactorización de servicios de API:** Implementar una jerarquía de captura y clasificación de errores descriptivos.
2. **Diseño de UI Resiliente:** Construir interfaces que gestionen estados de carga y errores con mecanismos de recuperación funcional.
3. **Defensa Asistida por IA:** Utilizar modelos de lenguaje para generar esquemas de validación de datos externos bajo normativas de documentación técnica.

1. Código Base del Proyecto (Punto de Partida)

Antes de aplicar medidas defensivas, debemos establecer nuestra base técnica. Este código representa una implementación "optimista" que carece de protección ante fallos externos.

Archivo: `api.js`

```
// Implementación básica vulnerable a fallos de red
export const getProducts = async () => {
  const response = await fetch('https://fakestoreapi.com/products/category/electronics');
  const data = await response.json();
  return data;
};
```

Archivo: `ProductCard.jsx`

```
import React from 'react';

const ProductCard = ({ product }) => {
  return (
    <div className="product-card">
      <img src={product.image} alt={product.title} width="150" />
      <h3>{product.title}</h3>
      <p>Precio: ${product.price}</p>
    </div>
  );
};

export default ProductCard;
```

Archivo: **App.jsx**

```
import React, { useState, useEffect } from 'react';
import { getProducts } from './api';
import ProductCard from './ProductCard';
import './App.css';

function App() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    getProducts()
      .then(data => {
        setProducts(data);
        setLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Cargando...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <div className="app-container">
      <h1>Catálogo de Electrónicos</h1>
      <div className="product-grid">
        {products.map(product => (
          <ProductCard key={product.id} product={product} />
        ))}
      </div>
    </div>
  );
}

export default App;
```

2. Paso 1: Refactorización de `api.js` para Clasificación de Errores

El primer pilar de la programación defensiva es no confiar en la respuesta del servidor. Debemos clasificar los fallos para que la UI pueda reaccionar de forma específica.

Instrucción: Sustituya el contenido de `api.js` por el siguiente bloque. Observe cómo validamos el estado HTTP antes de procesar el JSON.

```
export const getProducts = async () => {
  try {
    const response = await fetch('https://fakestoreapi.com/products/category/electronics');

    // response.ok verifica estados en el rango 200-299.
    // Cualquier otro valor requiere una gestión defensiva explícita.
    if (!response.ok) {
      if (response.status === 404) {
        throw { status: 404, message: "Recurso no encontrado (404)."; };
      } else if (response.status === 500) {
        throw { status: 500, message: "Error interno del servidor (500)."; };
      } else {
        throw { status: response.status, message: "Error de comunicación con la API." };
      }
    }

    const data = await response.json();
    return data;
  } catch (error) {
    // Captura defensiva de errores de red (ej. falta de conexión o DNS fallido)
    if (error instanceof TypeError) {
      throw { status: 0, message: "Fallo de red: Verifique su conexión a internet." };
    }
    // Propagamos el objeto de error enriquecido
    throw error;
  }
};
```

3. Paso 2: Estilos Defensivos en **App.css**

La resiliencia también se aplica al diseño visual. Utilizaremos propiedades modernas para evitar que fallos en el contenido rompan la estructura de la página.

Instrucción: Actualice **App.css** incluyendo un reset global y clases de retroalimentación visual centradas con Flexbox.

```
/* RESET DEFENSIVO: Control absoluto del modelo de caja */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box; /* Evita que el padding altere el ancho total del elemento */
}

.app-container {
  padding: 2rem;
  font-family: system-ui, -apple-system, sans-serif;
}

/* Grilla moderna con gap para evitar colapso de márgenes */
.product-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 20px; /* Sustituye márgenes manuales por espaciado constante */
}

/* Spinner centrado: Reservamos espacio para evitar Layout Shift (salto de página) */
.spinner-container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 50vh; /* Estrategia defensiva para reservar altura durante la carga */
}

.spinner {
  border: 4px solid rgba(0, 0, 0, 0.1);
  width: 40px;
  height: 40px;
  border-radius: 50%;
  border-left-color: #c51162;
  animation: spin 1s linear infinite;
}
```

```
@keyframes spin {  
  0% { transform: rotate(0deg); }  
  100% { transform: rotate(360deg); }  
}
```

```
/* Contenedor de error semántico y visible */
```

```
.error-container {  
  display: flex;  
  flex-direction: column;  
  justify-content: center;  
  align-items: center;  
  border: 2px solid #ff6347;  
  background-color: #fffafa;  
  padding: 40px;  
  margin: 20px auto;  
  border-radius: 12px;  
  max-width: 500px;  
  text-align: center;  
}
```

```
.btn-retry {  
  margin-top: 20px;  
  padding: 12px 24px;  
  background-color: #c51162;  
  color: white;  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
  font-weight: bold;  
}
```

```
.btn-retry:hover {  
  background-color: #a00e50;  
}
```

4. Paso 3: Interfaz de Usuario Resiliente en `App.jsx`

Evitaremos el uso de retornos tempranos (`return` condicionales) que bloquean la renderización completa. En su lugar, utilizaremos renderizado condicional con operadores lógicos para mantener una UI persistente.

Instrucción: Refactorice `App.jsx` utilizando `useCallback` para estabilizar referencias y evitar ciclos infinitos en el ciclo de vida del componente.

```
import React, { useState, useEffect, useCallback } from 'react';
import { getProducts } from './api';
import ProductCard from './ProductCard';
import './App.css';

function App() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  /**
   * MEMOIZACIÓN DEFENSIVA:
   * Utilizamos useCallback para asegurar que 'loadData' mantenga una referencia estable.
   * Esto previene que useEffect se dispare innecesariamente si el componente se re-renderiza,
   * evitando bucles infinitos y optimizando el consumo de la API.
   */
  const loadData = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      const data = await getProducts();
      setProducts(data);
    } catch (err) {
      // Acceso defensivo a propiedades de error
      setError(err.message || err.statusText || "Ocurrió un error inesperado.");
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    loadData();
  }, [loadData]); // Referencia estable gracias a useCallback
```

```

return (
  <div className="app-container">
    <h1>Tienda de Electrónicos Resiliente</h1>

    {/* Renderizado Condicional Defensivo */}
    {loading && (
      <div className="spinner-container">
        <div className="spinner"></div>
      </div>
    )}

    {!loading && error && (
      <div className="error-container">
        <p>⚠ Error detectado: {error}</p>
        <button className="btn-retry" onClick={loadData}>
          Reintentar Conexión
        </button>
      </div>
    )}

    {!loading && !error && (
      <div className="product-grid">
        {products.map(product => (
          <ProductCard key={product.id} product={product} />
        ))}
      </div>
    )}
  </div>
);
}

export default App;

```


5. Actividad con Inteligencia Artificial (Criterio 3.1.4)

Incluso con una respuesta HTTP 200 OK, los datos recibidos pueden ser inconsistentes (ej. precios negativos o URLs de imagen rotas). Como expertos, debemos automatizar la validación de estos esquemas.

Instrucción de Prompting: Redacte el siguiente prompt en un modelo de IA (ChatGPT, Claude o Gemini):

"Actúa como un experto en JavaScript y Programación Defensiva. Genera una función robusta llamada `validateProduct` que reciba un objeto `product`. La función debe retornar `true` solo si: el precio es un número mayor a cero, la URL de la imagen es válida y el título tiene al menos 5 caracteres. Incluye manejo de casos donde el objeto sea null o undefined."

6. Checklist de Evaluación

Antes de finalizar, asegúrese de cumplir con cada punto técnico:

- [] **Refactorización de API:** ¿El archivo `api.js` clasifica explícitamente errores 404, 500 y fallos de red (TypeError)?
- [] **Estabilidad del Modelo de Caja:** ¿Se incluyó el reset global `box-sizing: border-box` para prevenir errores de cálculo en dimensiones CSS?
- [] **Arquitectura de Hooks:** ¿Se implementó `useCallback` para `loadData` y se explicó su importancia para la estabilidad del componente?
- [] **UX de Resiliencia:** ¿El botón de reintento funciona correctamente cargando los datos sin refrescar el navegador?